SOFTWARE ENGINEERING

Chapter 4: Recommended Process Model

Based on: Software Engineering: A Practitioner's Approach (9th Edition)
Authors: Roger S. Pressman and Bruce R. Maxim

Student Name: Asal Ghafouri
Instructor: Engineer Rasoulzadeh
Course: Software Engineering

1. Introduction

After examining traditional process models and agile approaches, software engineering practitioners need practical guidance on how to organize real projects. Chapter 4 of Pressman and Maxim's book presents a recommended process model that combines the strengths of evolutionary development, prototyping, and agile principles.

No single process model fits every project. Teams must adapt a generic framework based on project size, risk, team skills, and customer needs. The recommended model emphasizes early stakeholder involvement, continuous feedback, evolutionary prototyping, and informed go/no-go decisions.

This chapter explains the recommended process model step by step and illustrates it with a practical example of developing an online shopping application.

2. Why a Recommended Process Model?

Every software project needs a road map. However, every project and every team is different. No single software engineering framework is appropriate for every product.

Developers and stakeholders should adapt generic process models and tailor them to fit the current project, the skills of the team members, and user needs.

The recommended process model is based on best industry practices and is designed to reduce risk while delivering value early.

3. Key Principles for Organizing Software Projects

The recommended approach is guided by several important principles:

1. It is risky to use a linear process model without ample feedback.
2. It is never possible nor desirable to plan big up-front requirements gathering.
3. Up-front requirements gathering may not reduce costs or prevent schedule slippage.
4. Appropriate project management is integral to software development.
5. Documents should evolve with the software and should not delay the start of construction.
6. Involve stakeholders early and frequently in the development process.
7. Testers should become involved in the process prior to software construction.

These principles encourage an iterative, feedback-driven approach rather than a rigid sequential plan.

4. Overview of the Recommended Process Model

The recommended process model is a hybrid that combines elements of the spiral model and agile development. It focuses on building and evaluating prototypes in successive cycles.

The main stages of the model are:

1. Requirements Definition
2. Preliminary Architectural Design
3. Resource Estimation
4. First Prototype Construction
5. Prototype Evaluation
6. Go / No-Go Decision
7. Prototype Evolution
8. Prototype Release
9. Maintain Released Software

Each cycle produces a working prototype that can be evaluated by stakeholders. Based on the evaluation, the team decides whether to continue, change direction, or stop the project.

5. Detailed Stages of the Recommended Process Model

5.1 Requirements Definition

The process begins by defining an initial set of requirements. Unlike traditional approaches, the goal is not to capture every detail at the beginning. Instead, the team identifies the most important features and creates a high-level understanding of the problem.

Requirements are expected to evolve as prototypes are built and evaluated.

Example:

For an online shopping application, the initial requirements might include: product browsing, user registration, shopping cart, and basic payment. Advanced features such as product recommendations or loyalty programs can be added later.

5.2 Preliminary Architectural Design

Before building the first prototype, the team creates a preliminary architecture. This architecture identifies major components, their relationships, and key technical decisions (for example, choice of platform, database, or communication style).

The architecture is intentionally lightweight so that it can be refined in later cycles.

Example:

The team decides to use a web-based architecture with a React frontend, a Node.js backend, and a relational database. Payment processing will be handled by a third-party service.

5.3 Resource Estimation

The team estimates the resources needed for the first prototype: people, time, tools, and budget. Estimation is intentionally limited to the current cycle rather than the entire project.

This reduces the risk of inaccurate long-term predictions.

Example:

The team estimates that the first prototype (product catalogue and basic cart) will require three developers working for three weeks, plus one designer and one tester.

5.4 First Prototype Construction

The team builds a working prototype that demonstrates the most important features. The prototype does not need to be complete or perfect. Its main purpose is to make the product tangible so that stakeholders can provide meaningful feedback.

Example:

The first prototype of the shopping application allows users to browse products, add items to a cart, and view the total price. Payment and order tracking are not yet implemented.

5.5 Prototype Evaluation

Stakeholders (customers, users, and managers) evaluate the prototype. They provide feedback on usability, missing features, performance, and business value.

Evaluation is a critical activity because it drives the direction of the next cycle.

Example:

During evaluation, customers request a faster search function and the ability to save favorite products. They also suggest simplifying the cart interface.

5.6 Go / No-Go Decision

After evaluation, the team and stakeholders make a conscious decision:

- Go – Continue development with the current direction.
- No-Go – Stop the project or significantly change direction.

This decision point helps control risk and prevents wasted effort on products that no longer make sense.

Example:

After reviewing the first prototype, the stakeholders decide to continue (Go) but to prioritize search improvement and a simpler cart in the next cycle.

5.7 Prototype Evolution

If the decision is to continue, the team plans the next prototype. This includes:

- Defining the new scope (what features will be added or improved).
- Constructing the new prototype.
- Testing the new prototype.

Each evolution cycle produces a more complete and refined version of the software.

Example:

In the second cycle, the team improves product search, simplifies the shopping cart, and adds user registration. In later cycles they add payment and order tracking.

5.8 Prototype Release

When a prototype reaches an acceptable level of quality and functionality, it can be released to real users. Release may be limited (for example, a pilot group) or public.

Release provides valuable real-world feedback that feeds into future evolution cycles.

5.9 Maintain Released Software

After release, the software enters a maintenance phase. The team continues to fix defects, improve performance, and add new features based on user feedback. Maintenance is treated as a continuation of the evolutionary process rather than a separate activity.

6. Practical Example: Online Shopping Application

To illustrate the recommended process model, consider the development of an online shopping application.

6.1 Project Goal

The company wants a web application that allows customers to browse products, create accounts, manage a shopping cart, make payments, and track orders.

6.2 Cycle 1 – First Prototype

Requirements: Product catalogue and basic shopping cart.
Architecture: Web application with frontend, backend, and database.
Construction: Developers build a working prototype with product listing and cart functionality.
Evaluation: Stakeholders request better search and a cleaner cart interface.
Decision: Go – Continue with improvements in the next cycle.

6.3 Cycle 2 – Evolution

New Scope: Improved search, simplified cart, and user registration.
Construction & Testing: The team implements the new features and performs basic testing.
Evaluation: Stakeholders are satisfied and request online payment in the next cycle.

6.4 Cycle 3 – Further Evolution

New Scope: Online payment integration and order records.
Construction & Testing: Payment service is integrated and multiple payment scenarios are tested.
Decision: Go – Prepare a limited release.

6.5 Release and Maintenance

The application is released to a group of pilot users. Feedback from real usage leads to further improvements such as order tracking and performance optimization. The evolutionary cycle continues after release.

7. Advantages of the Recommended Process Model

1. Early and continuous feedback from stakeholders.
2. Reduced risk through go/no-go decision points.
3. Flexibility to adapt when requirements change.
4. Working software is available early for evaluation.
5. Documentation evolves with the product instead of delaying construction.
6. Suitable for projects with uncertain or evolving requirements.

8. Challenges and Limitations

1. Requires active and continuous stakeholder participation.
2. Teams must be skilled in iterative development and prototyping.
3. Estimation remains difficult, especially for later cycles.
4. Without discipline, the process can become unstructured.
5. Not ideal for projects with strict regulatory or contractual constraints that demand extensive upfront documentation.

9. When Is This Model Suitable?

The recommended process model is particularly useful when:

- Requirements are not fully known at the beginning.
- Customer feedback is essential for success.
- The project can be delivered in evolutionary increments.
- The team can communicate regularly with stakeholders.
- Risk reduction and early validation are important.

For highly regulated systems or projects with fixed, well-understood requirements, a more traditional or hybrid approach with stronger documentation may still be appropriate.

10. Conclusion

Chapter 4 of Pressman and Maxim's book presents a practical recommended process model that combines evolutionary prototyping with agile principles. The model encourages early stakeholder involvement, continuous feedback, and conscious go/no-go decisions.

By building and evaluating prototypes in successive cycles, teams can reduce risk, deliver value early, and adapt to changing requirements. The online shopping application example shows how the model can be applied in a realistic project setting.

Successful use of the recommended process model depends on disciplined teamwork, active customer participation, and a willingness to adapt the process to the specific needs of each project.

11. References

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Boehm, B. W. (1988). A spiral model of software development and enhancement. *IEEE Computer*, 21(5), 61–72.

Beck, K., et al. (2001). *Manifesto for Agile Software Development*. https://agilemanifesto.org/